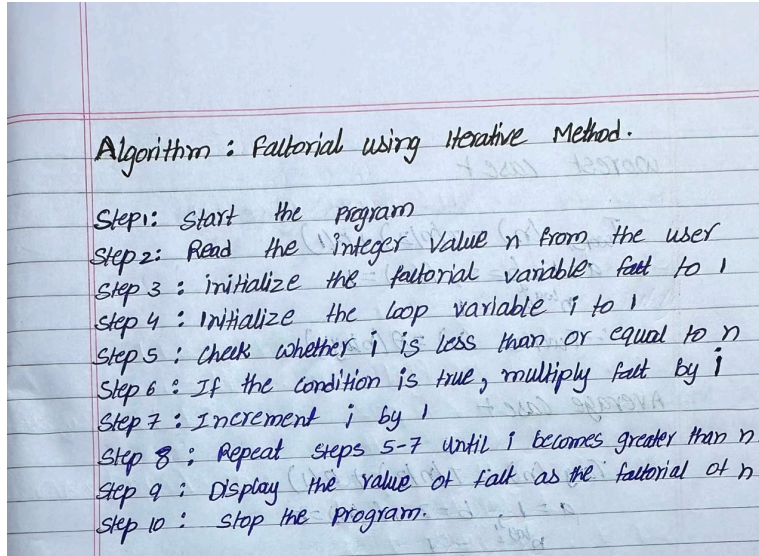


PRACTICAL - 4

AIM : Implementation and Time analysis of factorial program using iterative and recursive method

ALGORITHM // Iterative method



Algorithm : Factorial using Iterative Method.

- Step 1: Start the program
- Step 2: Read the integer value n from the user
- Step 3: Initialize the factorial variable $fact$ to 1
- Step 4: Initialize the loop variable i to 1
- Step 5: Check whether i is less than or equal to n
- Step 6: If the condition is true, multiply $fact$ by i
- Step 7: Increment i by 1
- Step 8: Repeat steps 5-7 until i becomes greater than n .
- Step 9: Display the value of $fact$ as the factorial of n
- Step 10: Stop the program.

PROGRAM :

```
#include <iostream>
using namespace std;

long long factorial(int n) {
    long long result = 1;

    for (int i = 1; i <= n; i++) {
        result = result * i;

        cout << "Step " << i << ": "
              << result << endl;
    }

    return result;
}

int main() {
    int n;

    cout << "Enter a number: ";
    cin >> n;
```



```
cout << "\n--- Iterative Process ---" << endl;  
  
long long result = factorial(n);  
  
cout << "\nFactorial of " << n << " = " << result << endl;  
  
return 0;  
}
```

OUTPUT :

```
Enter a number: 5  
  
--- Iterative Process ---  
Step 1: 1  
Step 2: 2  
Step 3: 6  
Step 4: 24  
Step 5: 120  
  
Factorial of 5 = 120
```



TIME COMPLEXITY :

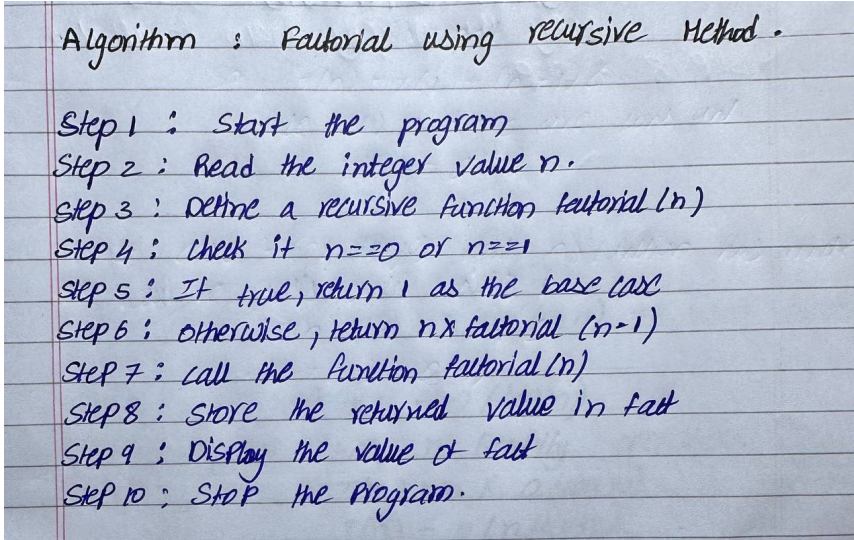
Time complexity
iterative factorial

```
int factorial (int n)
{
    int fact = 1;
    for (int i = 1; i <= n; i++)
        fact = fact * i;
    return fact;
}
```

The for loop runs n times
 $i++ \rightarrow$ constant time
 comparison $\rightarrow$ constant time
 $fact = fact * i \rightarrow$ constant time
 So, each iteration takes constant time C .
 $T(n) = C + C + C + \dots + C$
 for n iterations
 $T(n) = Cn$
 $\therefore T(n) = O(n)$ or $T(n) = \Theta(n)$

Best, Average and worst case.
 for iterative factorial the loop always runs according to n . There is no early termination based on the input value.

Best = worst = Average = $O(n)$
 $T(n) = C + C + C + \dots + C$
 $T(n) = Cn$
 $T(n) = O(n)$

ALGORITHM // Recursive method

Algorithm : Factorial using recursive Method.

Step 1 : Start the program
Step 2 : Read the integer value n.
Step 3 : Define a recursive function factorial(n)
Step 4 : Check if $n == 0$ or $n == 1$
Step 5 : If true, return 1 as the base case
Step 6 : Otherwise, return $n \times \text{factorial}(n-1)$
Step 7 : Call the function factorial(n)
Step 8 : Store the returned value in fact
Step 9 : Display the value of fact
Step 10 : Stop the program.

PROGRAM :

```
#include <iostream>
using namespace std;

long long factorial(int n) {
    // Base case
    if (n == 0 || n == 1) {
        cout << "factorial(" << n << ") = 1" << endl;
        return 1;
    }

    // Show the recursive call
    cout << "factorial(" << n << ") = "
        << n << " * factorial(" << n - 1 << ")" << endl;

    // Recursive call
    long long result = n * factorial(n - 1);

    // Show the result after returning
    cout << "factorial(" << n << ") = "
        << result << endl;

    return result;
}

int main() {
    int n;
    cout << "Enter a number: ";
```

```
cin >> n;  
    cout << "\n--- Recursive Process ---" << endl;  
  
    long long result = factorial(n);  
  
    cout << "\nFactorial of " << n << " = " << result << endl;  
  
    return 0;  
}
```

OUTPUT :

```
Enter a number: 5  
  
--- Recursive Process ---  
factorial(5) = 5 * factorial(4)  
factorial(4) = 4 * factorial(3)  
factorial(3) = 3 * factorial(2)  
factorial(2) = 2 * factorial(1)  
factorial(1) = 1  
factorial(2) = 2  
factorial(3) = 6  
factorial(4) = 24  
factorial(5) = 120  
  
Factorial of 5 = 120  
  
=== Code Execution Successful ===
```



TIME COMPLEXITY :

Time Complexity

1. Recursive Factorial

```
int factorial (int n)
{
    if (n == 0 || n == 1)
        return 1;
    return n * factorial (n-1);
}
```

For every call :
factorial (n)
it makes only one recursive call :
factorial (n-1)
Then : $n * \text{factorial (n-1)}$
takes constant time :
 $T(n) = T(n-1) + C$
where C is constant
By using Master's Theorem:
 $T(n) = aT(n/b) + P(n)$
where $a \geq 1$ & $b > 1$
for factorial, $T(n-1) + C$
we cannot use Master's Theorem
By using recursive expansion
 $T(n) = T(n-1) + C$
 $T(n-1) = T(n-2) + C$
 $T(n) = T(n-2) + 2C$
 $T(n) = T(n-3) + 3C$
 $T(n) = T(n-k) + kC$
At the base case : $n-k=1$
 $k=n-1$
So $T(n) = T(1) + (n-1)C$
since $T(1)$ & C are constants!



$$\begin{aligned}T(n) &= C_1 + C_2 n \\T(n) &= O(n) \\T(n) &= O(n)\end{aligned}$$

Best case +

$$\begin{aligned}n &= 0 \text{ or } n = 1 \text{ returns immediately} \\T(n) &= C \\T(n) &= O(1)\end{aligned}$$

Average case +

$$\begin{aligned}T(n) &= T(n-1) + C \\T(n) &= T(n-2) + 2C \\T(n) &= T(n-3) + 3C \\T(n) &= T(n-k) + kC \\n - k &= 1 \\k &= n - 1\end{aligned}$$

So, $T(n) = T(1) + (n-1)C$
Since $T(1)$ and C are constant
 $T(n) = O(n)$

Worst case + For larger input n , function has to make all the recursive calls until it reaches 1.

$$\begin{aligned}T(n) &= T(n-1) + C \\T(n) &= O(n)\end{aligned}$$

CONCLUSION :

The factorial program was successfully implemented using both **iterative and recursive methods**. The iterative method calculates the factorial using a loop, while the recursive method calculates it by repeatedly calling the same function with a smaller value.

From the time analysis, both methods have a **time complexity of $O(n)$** . However, the iterative method generally requires less memory because the recursive method uses additional stack space for each function call. Therefore, the **iterative approach is more memory-efficient**, while the recursive approach provides a simpler and more intuitive representation of the factorial concept.